

Wazuh Sysmon PowerShell Detection – Decoder Validation & Rule Tuning (Encoded Command Detection)

Project Overview

This project focuses on **validating Sysmon Event ID 1 ingestion**, **verifying correct field decoding**, and **tuning Wazuh detection rules** to accurately identify **malicious PowerShell activity**, specifically **base64 encoded commands**, while avoiding unnecessary noise.

The goal was not to create new decoders from scratch, but to **prove end-to-end visibility**:

PowerShell execution → Sysmon logging → Wazuh decoding → Alert generation → Dashboard verification.

Objectives

- Confirm Sysmon Event ID 1 logs are **correctly decoded by Wazuh**
- Validate **commandLine field availability** for PowerShell detection
- Tune detection rules for **EncodedCommand abuse**
- Ensure alerts trigger with correct **rule ID, MITRE mapping, and severity**
- Maintain SOC-friendly signal-to-noise balance

Why This Project Matters

PowerShell is one of the **most abused attack vectors** in real-world intrusions.

Attackers often use:

- -EncodedCommand
- Base64 obfuscation
- LOLBins chaining

Without **proper decoding + rule alignment**, these attacks:

- Either generate **no alerts**
- Or get buried under noise

This project ensures **detections are both accurate and actionable**.

Tools & Stack Used

- **Wazuh Manager (4.x)**
- **Wazuh Windows Agent**
- **Sysmon (SwiftOnSecurity config)**
- **PowerShell**
- **Wazuh Dashboard / Discover**
- **wazuh-logtest**

Step-by-Step Implementation

Step 1: Sysmon Configuration on Windows

- Sysmon installed with a **focused configuration**
- Event ID 1 (Process Create) enabled
- Ensured CommandLine logging was active

Purpose:

To guarantee PowerShell execution details are logged at the endpoint.

```
<syscheck>

  <disabled>no</disabled>

  <!-- Malware-prone user and temp locations -->
  <directories check_all="yes" realtime="yes">C:\Users</directories>
  <directories check_all="yes" realtime="yes">C:\Windows\Temp</directories>

  <!-- ProgramData monitored, NOT realtime -->
  <directories check_all="yes">C:\ProgramData</directories>

  <!-- Phase 1 ignores -->
  <ignore>C:\ProgramData\Microsoft\Windows\CapabilityAccessManager</ignore>
  <ignore type="sregex">\.db$</ignore>
  <ignore type="sregex">\.db-wal$</ignore>
  <ignore type="sregex">\.db-shm$</ignore>
  <ignore>C:\ProgramData\Microsoft\Diagnosis</ignore>
  <ignore>C:\ProgramData\Microsoft\Windows\WER</ignore>
  <ignore>C:\Windows\SoftwareDistribution</ignore>
```

Step 2: PowerShell Execution Test (Baseline)

powershell.exe -ExecutionPolicy Bypass -Command Get-Date

Purpose:

To confirm:

- Sysmon captures execution
- Wazuh decodes the event
- No false high-severity alert is generated

```
PS C:\Users\Abdul Rehman> powershell.exe -ExecutionPolicy Bypass -Command Get-Date
18 January 2026 23:45:12
```

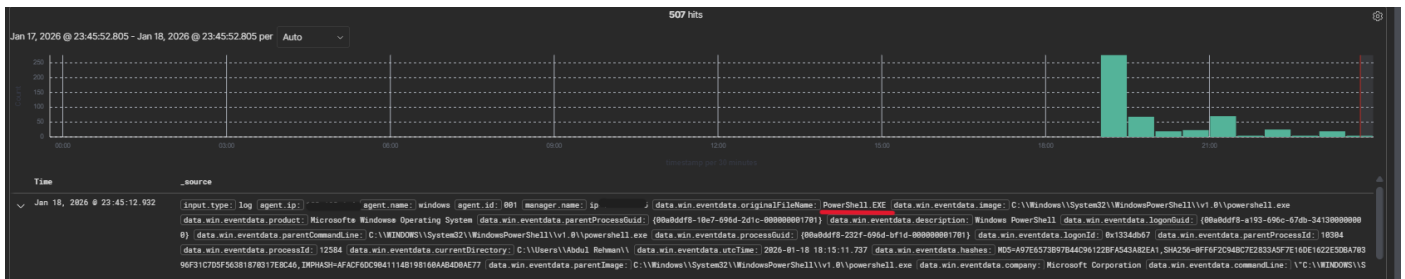
Step 3: Event Discovery in Wazuh Dashboard

What was done:

Checked **Wazuh Dashboard** → **Discover** to locate the PowerShell execution event generated from the test.

Purpose:

To confirm Sysmon logs are ingested and the commandLine field is **correctly decoded and visible**.



```
data.win.system.message
{
  "Process Create": {
    "RuleName": "Process Create",
    "UtcTime": "2026-01-18 15:11:737",
    "ProcessGuid": "{00a0ddf8-232f-696d-bf1d-000000001701}",
    "ProcessId": 12584,
    "Image": "C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe",
    "FileVersion": "10.0.26100.5074 (WinBuild.160101.0800)",
    "Description": "Windows PowerShell",
    "Product": "Microsoft Windows Operating System",
    "Company": "Microsoft Corporation",
    "OriginalFileName": "PowerShell.EXE",
    "CommandLine": "\"C:\\WINDOWS\\System32\\WindowsPowerShell\\v1.0\\powershell.exe\" -ExecutionPolicy Bypass -Command Get-Date",
    "CurrentDirectory": "C:\\Users\\Abdul Rehman\\",
    "User": "Abdul Rehman",
    "LogonGuid": "{00a0ddf8-a193-696c-67db-341380000000}",
    "LogonId": 0x1334D867,
    "TerminalSessionId": 4,
    "IntegrityLevel": "High",
    "Hashes": "MD5-A97E6573B97B44C96122BFA543A82EA1, SHA256-0FF6F2C94BC7E283A5F7E16DE1622E5DBA78396F31C7D5F56381870317E8C46, IMPHASH-AFACF6DC9041114B198160AAB4D0AE77",
    "ParentProcessGuid": "{00a0ddf8-10e7-696d-2d1c-000000001701}",
    "ParentProcessId": 18384,
    "ParentImage": "C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe",
    "ParentCommandLine": "C:\\WINDOWS\\System32\\WindowsPowerShell\\v1.0\\powershell.exe",
    "ParentUser": "Abdul Rehman"
  }
}
```

Step 4: Decoder Validation Using Rule Test

What was done:

Copied the PowerShell Sysmon event from Discover and tested it in **Wazuh Ruleset Test**.

Purpose:

To verify the event passes **Phase 2 decoding** and matches the expected Windows/Sysmon decoder.

Ruleset Test

```
"Process Create": {
  RuleName: "Process Create",
  UtcTime: "2026-01-18 15:11:737",
  ProcessGuid: "{00a0ddf8-232f-696d-bf1d-000000001701}",
  ProcessId: 12584,
  Image: "C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe",
  FileVersion: "10.0.26100.5074 (WinBuild.160101.0800)",
  Description: "Windows PowerShell",
  Product: "Microsoft Windows Operating System",
  Company: "Microsoft Corporation",
  OriginalFileName: "PowerShell.EXE",
  CommandLine: "\"C:\\WINDOWS\\System32\\WindowsPowerShell\\v1.0\\powershell.exe\" -ExecutionPolicy Bypass -Command Get-Date",
  CurrentDirectory: "C:\\Users\\Abdul Rehman\\",
  User: "Abdul Rehman",
  LogonGuid: "{00a0ddf8-a193-696c-67db-341380000000}",
  LogonId: 0x1334D867,
  TerminalSessionId: 4,
  IntegrityLevel: "High",
  Hashes: "MD5-A97E6573B97B44C96122BFA543A82EA1, SHA256-0FF6F2C94BC7E283A5F7E16DE1622E5DBA78396F31C7D5F56381870317E8C46, IMPHASH-AFACF6DC9041114B198160AAB4D0AE77",
  ParentProcessGuid: "{00a0ddf8-10e7-696d-2d1c-000000001701}",
  ParentProcessId: 18384,
  ParentImage: "C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe",
  ParentCommandLine: "C:\\WINDOWS\\System32\\WindowsPowerShell\\v1.0\\powershell.exe",
  ParentUser: "Abdul Rehman"
}
```

D. Test

```
WARNING: (7619): Empty 'if_sid' value. Rule '80780' will be ignored.
WARNING: (7616): List 'etc/lists/audit-keys' could not be loaded. Rule '80780' will be ignored.
WARNING: (7618): Group 'audit_watch_write' was not found. Invalid 'if_group'. Rule '80780' will be ignored.
WARNING: (7618): Group 'audit_watch_write' was not found. Invalid 'if_group'. Rule '80780' will be ignored.
WARNING: (7616): List 'etc/lists/audit-keys' could not be loaded. Rule '80780' will be ignored.
WARNING: (7619): Empty Mitre information for rule '180300'.
WARNING: (7619): Empty Mitre information for rule '180300'.
WARNING: (7619): Empty Mitre information for rule '180300'.
WARNING: (7619): Empty Mitre information for rule '180300'.
WARNING: (7619): Empty Mitre information for rule '180300'.
WARNING: (7612): Rule ID '180300' is duplicated. Only the first occurrence will be considered.
WARNING: (7612): Rule ID '180300' is duplicated. Only the first occurrence will be considered.
WARNING: (7612): Rule ID '180300' is duplicated. Only the first occurrence will be considered.
WARNING: (7615): Invalid 'if_matched_sid' value 'system.event'. Rule '110001' will be ignored.
INFO: (7602): Session initialized with token 'testToken'

**Phase 1: Completed pre-decoding.
Full event: "Process Create."

**Phase 2: Completed decoding.
No decoder matched.

**Phase 1: Completed pre-decoding.
Full event: "Process Create."

**Phase 2: Completed decoding.
No decoder matched.

**Phase 1: Completed pre-decoding.
Full event: "UtcTime: '2026-01-18 15:11:737'"

**Phase 2: Completed decoding.
```

```
**Phase 2: Completed decoding.  
No decoder matched.  
  
**Phase 1: Completed pre-decoding.   
full event: 'CommandLine: "C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe" -ExecutionPolicy Bypass -Command Get-Date'  
  
**Phase 2: Completed decoding.  
No decoder matched.  
  
**Phase 1: Completed pre-decoding.  
full event: 'CurrentDirectory: C:\Users\Abdul Rehman\  
  
**Phase 2: Completed decoding.   
No decoder matched.  
  
**Phase 1: Completed pre-decoding.  
full event: 'User: LAPTOP-20EH9HNF\Abdul Rehman'  
  
**Phase 2: Completed decoding.  
No decoder matched.  
  
**Phase 1: Completed pre-decoding.  
full event: 'LogonGuid: {00a0ddf8-a193-696c-67db-341300000000}'  
  
**Phase 2: Completed decoding.  
No decoder matched.  
  
**Phase 1: Completed pre-decoding.  
full event: 'LogonId: 0x13340867'  
  
**Phase 2: Completed decoding.  
No decoder matched.
```

Step 5: Custom PowerShell Rule Addition (Manager Side)

What was done:

Added custom PowerShell detection rules in local_rules.xml on the Wazuh manager.

Purpose:

To detect **encoded commands**, **LOLBins**, and **suspicious PowerShell behavior** with proper severity and MITRE mapping.

```

</group>

<group name="sysmon,windows,powershell,phase3">

  <!-- CRITICAL: Encoded PowerShell -->
  <rule id="300010" level="12">
    <if_sid>92027</if_sid>
    <match>(?!i)commandline:.*(-enc|-encodedcommand)</match>
    <description>CRITICAL: Encoded PowerShell command execution</description>
    <mitre>T1027,T1059.001</mitre>
    <group>windows,powershell,obfuscation,mitre</group>
  </rule>

  <!-- PowerShell Download / IEX -->
  <rule id="300020" level="11">
    <if_sid>92027</if_sid>
    <match>(?!i)commandline:.*(invoke-webrequest|iwr|downloadstring|wget|curl|iex)</match>
    <description>Suspicious PowerShell download or remote execution</description>
    <mitre>T1105,T1059.001</mitre>
    <group>windows,powershell,download,mitre</group>
  </rule>

  <!-- PowerShell LOLBins -->
  <rule id="300030" level="13">
    <if_sid>92027</if_sid>
    <match>(?!i)commandline:.*(rundll32|mshta|certutil|regsvr32)</match>
    <description>CRITICAL: PowerShell spawning LOLBins</description>
    <mitre>T1218,T1059.001</mitre>
    <group>windows,powershell,lolbins,mitre</group>
  </rule>

  <!-- Baseline PowerShell -->
  <rule id="300001" level="3">
    <if_sid>92027</if_sid>
    <description>Baseline PowerShell execution</description>
    <group>windows,powershell,baseline</group>
    <options>no_full_log</options>
  </rule>

</group>

```

Step 6: Encoded PowerShell Execution Test

What was done:

Executed a **base64 encoded PowerShell command** on the Windows endpoint.

Purpose:

To simulate a real attacker technique and validate detection of **obfuscated PowerShell execution**.

```

PS C:\Users\Abdul Rehman> $cmd = "Get-Date"
PS C:\Users\Abdul Rehman> [Convert]::ToBase64String([System.Text.Encoding]::Unicode.GetBytes($cmd))
RwBLAHQALQBEGEAdABlAA==
PS C:\Users\Abdul Rehman> powershell -EncodedCommand RwBLAHQALQBEGEAdABlAA==

18 January 2026 23:23:12

```

Step 7: Alert Verification in Wazuh Dashboard

What was done:

Reviewed the generated alert in the Wazuh dashboard after rule execution.

Purpose:

To confirm the alert triggered with the **correct rule ID, severity level, and MITRE technique (T1059.001)**.

Jan 17, 2026 @ 23:55:03.815 - Jan 18, 2026 @ 23:55:03.815 per Auto

Time

Jan 18, 2026 @ 23:23:13.592

input.type: log agent.ip: agent.name: windows agent.id: 001 manager.name: data.win.eventdata.originalFileName: PowerShell.EXE data.win.eventdata.image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

data.win.eventdata.product: Microsoft Windows Operating System data.win.eventdata.parentProcessGuid: {00a0ddfb-10e7-696d-2d1c-000000001701} data.win.eventdata.description: Windows PowerShell data.win.eventdata.logonGuid: {00a0ddfb-a193-696c-67db-341300000000}

data.win.eventdata.parentCommandLine: C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe data.win.eventdata.processGuid: {00a0ddfb-1e08-696d-3c1d-000000001701} data.win.eventdata.logonId: 0x1334db07 data.win.eventdata.parentProcessId: 16384

data.win.eventdata.processId: 2228 data.win.eventdata.currentDirectory: C:\Users\Abdul Rehman\ data.win.eventdata.uptime: 2026-01-18 17:53:12.409 data.win.eventdata.hashes: MD5-A97E6573B97B4C96122BF4543A2EA1 SHA256-BFF6F2C94BC7E283A5F7E1A0E1622E30BA78396F31C7D5 F563B1878317EBC46_INPHASH-AFAC6DC9B4114B198168AB4DAE77 data.win.eventdata.parentImage: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe data.win.eventdata.company: Microsoft Corporation data.win.eventdata.commandLine: C:\WINDOWS\System32\Windows

Expanded document

View surrounding documents View single document

JSON

index	wazuh-alerts-4.x-2026.01.18
agent.id	001
agent.ip	
agent.name	windows
data.win.eventdata.commandLine	C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe -EncodedCommand R01ANQUR0RFA0A0A0A0A0
data.win.eventdata.company	Microsoft Corporation
data.win.eventdata.currentDirectory	C:\Users\Abdul Rehman\
data.win.eventdata.description	Windows PowerShell
data.win.eventdata.fileVersion	10.0.26100.5074 (WinBuild.160801.0800)
data.win.eventdata.hashes	MD5-A97E6573B97B4C96122BF4543A2EA1 SHA256-BFF6F2C94BC7E283A5F7E1A0E1622E30BA78396F31C7D5 F563B1878317EBC46_INPHASH-AFAC6DC9B4114B198168AB4DAE77
data.win.eventdata.image	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
data.win.eventdata.integrityLevel	High
data.win.eventdata.logonGuid	{00a0ddfb-a193-696c-67db-341300000000}
data.win.eventdata.logonId	0x1334db07
data.win.eventdata.originalFileName	PowerShell.EXE

data.win.system.opcode	0
data.win.system.processID	6288
data.win.system.providerGuid	{5770385f-c22a-43e0-bf4c-06f5698ffbd9}
data.win.system.providerName	Microsoft-Windows-Sysmon
data.win.system.severityValue	INFORMATION
data.win.system.systemTime	2026-01-18T17:53:12.4279931Z
data.win.system.task	1
data.win.system.threadID	8400
data.win.system.version	5
decoder.name	windows_eventchannel
id	1768758793.1043145
input.type	log
location	EventChannel
manager.name	
rule.description	Powershell.exe spawned a powershell process which executed a base64 encoded command
rule.firedtimes	1
rule.groups	sysmon, sysmon_eid1_detections, windows
rule.id	92057
rule.level	12
rule.mail	true
rule.mitre.id	T1059.001
rule.mitre.tactic	Execution
rule.mitre.technique	PowerShell
timestamp	Jan 18, 2026 @ 23:23:13.592

Skills Gained

- Validated **Sysmon Event ID 1** ingestion and decoding in Wazuh
- Verified **PowerShell commandLine** field extraction
- Tuned **custom Wazuh rules** for encoded PowerShell detection
- Used **Ruleset Test** to safely validate decoder and rule matches
- Analyzed alerts in **Wazuh Discover** with rule ID, level, and MITRE mapping
- Improved **signal-to-noise balance** using baseline vs high-risk rules

🏁 Conclusion

This project confirms that **PowerShell encoded command activity is correctly decoded, detected, and alerted** using Sysmon and Wazuh. By validating telemetry first and tuning rules on real endpoint events, high-confidence alerts were achieved without adding noise. The workflow reflects real SOC detection engineering practices.